MARMARA UNIVERSITY ENGINEERING FACULTY

EE 4065

Introduction to Embedded Digital Image Processing

Homework 6 Report

NAME:	*Baran*	*Muhammet Yücel*
SURNAME:	*ORMAN*	*ÇELİK*
NUMBER:	*150721063*	*150721024*

QUESTION

Handwritten Digit Recognition from Digital Images (Application 13.7)

1. Introduction

This study presents the design and implementation of a handwritten digit recognition system deployed on an STM32 microcontroller. The objective was to evaluate how modern convolutional neural network architectures behave when transferred from a high-performance Python environment to a highly resource-constrained embedded platform. In order to achieve this, four state-of-the-art deep learning models, namely SqueezeNet, EfficientNet-B0, MobileNetV2 and ResNet50, were trained on the MNIST dataset, converted to INT8 quantized TensorFlow Lite format, and then embedded into the STM32 using the X-CUBE-AI framework. While SqueezeNet and EfficientNet-B0 achieved strong performance in Python, they could not be deployed on the STM32 due to memory constraints. MobileNetV2 and ResNet50 were successfully executed on the microcontroller and evaluated through real-time hardware-in-the-loop testing.

2. Dataset and Preprocessing

The MNIST dataset, which consists of 70,000 grayscale images of handwritten digits ranging from 0 to 9 with a resolution of 28×28 pixels, was used in this work. Since the selected convolutional neural networks were originally designed for three-channel color images with larger input sizes, each MNIST image was first normalized and then converted to a 32×32 RGB format. This conversion was achieved through bilinear interpolation, which preserves spatial smoothness while resizing, followed by replication of the grayscale values across three channels. The same preprocessing pipeline was applied both during Python training and inside the STM32 firmware in order to ensure consistency between training and deployment.

3. Python-Based Model Training

All four convolutional neural networks were trained in Python using TensorFlow and Keras. SqueezeNet and EfficientNet-B0 were selected because of their reputation for achieving high accuracy with relatively low computational cost, while MobileNetV2 was chosen specifically for embedded and mobile platforms. ResNet50 was included as a deeper and more computationally intensive reference model. Each network was trained on the MNIST training set and evaluated on the MNIST test set. After training, confusion matrices were computed to analyze class-wise performance and identify digit-specific misclassifications. These Python-side results represent the upper bound of performance, since they are obtained without any hardware or quantization limitations.

4. Python-Based Model Results

Confusion matrix of the EfficientNet-B0 model trained on the MNIST dataset. The diagonal elements represent correctly classified digits, while off-diagonal elements indicate misclassifications between digit classes. The matrix shows that EfficientNet-B0 achieves strong

recognition performance, with most errors occurring between visually similar digits such as 2–3, 4–9, and 8–9.

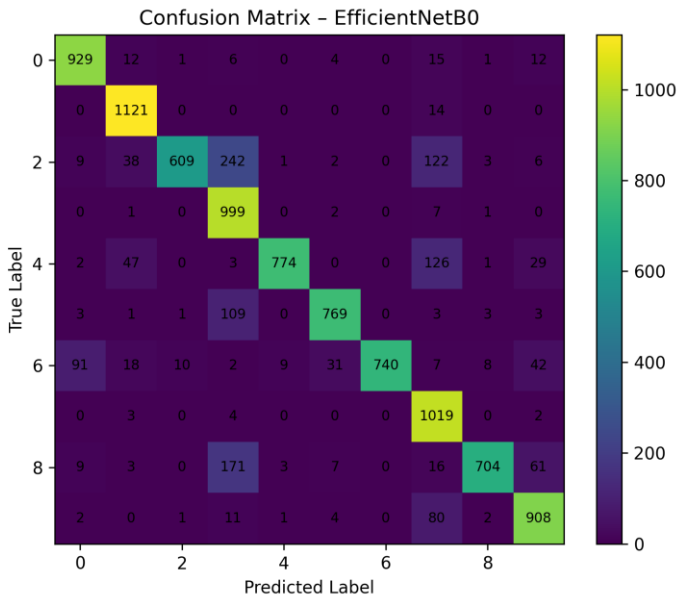

Figure 1 – EfficientNet-B0 Confusion Matrix

Confusion matrix of the MobileNetV2 model on the MNIST test set. Compared to EfficientNet-B0 and ResNet50, MobileNetV2 exhibits higher confusion across multiple digit pairs, particularly between digits 2, 3, 4, 7, and 8. This reflects the reduced representational capacity of MobileNetV2, which was designed for lightweight and embedded-oriented inference.

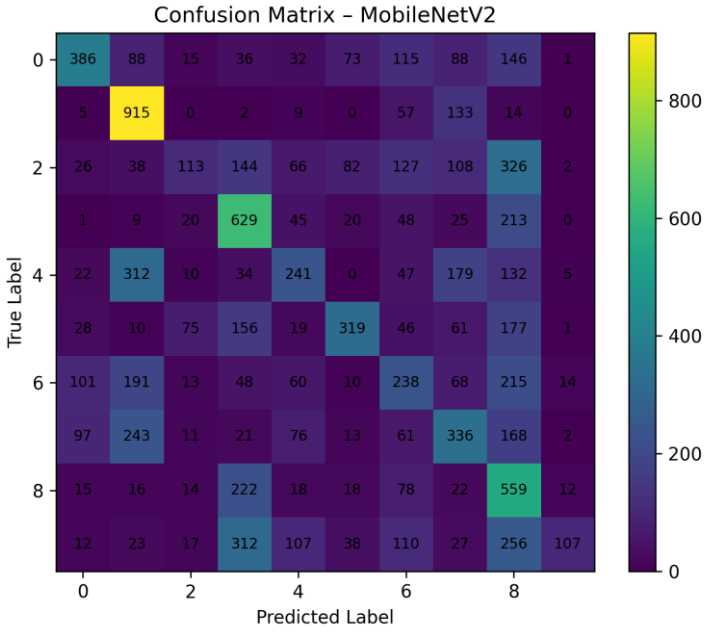

Figure 2 - MobileNetV2 Confusion Matrix

Confusion matrix of the ResNet50 model trained on MNIST. The matrix shows strong diagonal dominance, indicating high classification accuracy. Minor confusions appear primarily between visually similar digits such as 5–6, 8–9, and 0–9. Overall, ResNet50 provides the highest baseline accuracy among the tested models before deployment on the STM32 platform.

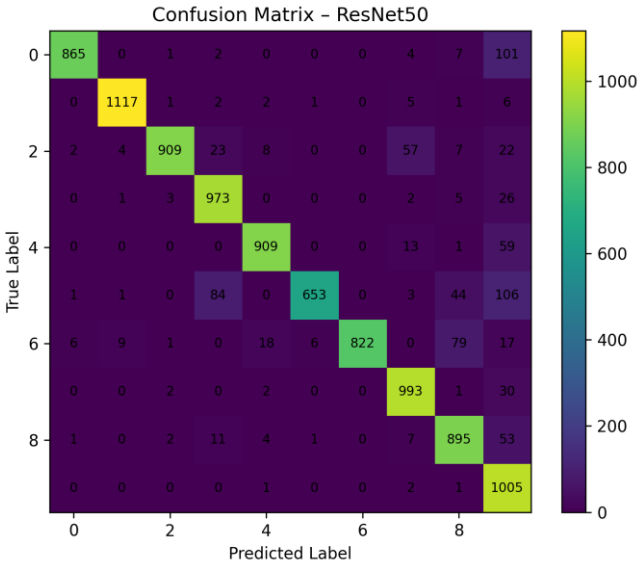

Figure 3 – ResNet50 Confusion Matrix

Confusion matrix of the SqueezeNet model on the MNIST dataset. Despite its compact architecture, SqueezeNet achieves high classification accuracy with relatively few misclassifications. Errors mainly occur between digits with similar stroke structures, such as 3–5 and 8–9, demonstrating the trade-off between model compactness and representational capacity.

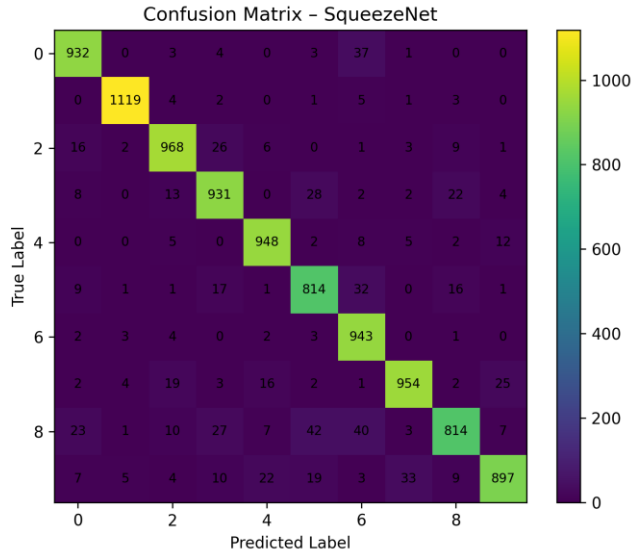

Figure 4 - SqueezeNet Confusion Matrix

5. Quantization and TensorFlow Lite Conversion

To enable deployment on the STM32 microcontroller, all trained models were converted to TensorFlow Lite format using INT8 quantization. During this process, representative datasets were used to calibrate the dynamic ranges of the network activations and weights. Quantization drastically reduces memory usage and computational complexity by replacing floating-point operations with fixed-point arithmetic, which is essential for running deep neural networks on microcontrollers. The resulting .tflite files represent the optimized embedded versions of the original Python models.

6. STM32 Deployment Using X-CUBE-AI

The quantized TensorFlow Lite models were imported into STM32CubeIDE through the X-CUBE-AI middleware. The tool generated optimized C code for each neural network and allocated the necessary memory buffers for activations, inputs and outputs. Due to memory limitations of the STM32F446RE platform, only MobileNetV2 and ResNet50 could be fully compiled and executed on the target device. SqueezeNet and EfficientNet-B0 exceeded the available memory and were therefore evaluated only in Python. The embedded firmware was designed to receive MNIST images from a PC via UART, preprocess them on the microcontroller, run neural network inference and return the predicted digit back to the PC.

7. Hardware-in-the-Loop Testing Framework

A custom Python testing framework was developed to evaluate the STM32 models in real time. In each test iteration, a random MNIST image was selected and transmitted to the STM32 over a serial connection. The microcontroller performed the same preprocessing and inference as during training and returned the predicted digit to the PC. Python then compared the STM32 prediction with the ground-truth label and recorded the result. This approach enabled the measurement of both classification accuracy and inference latency directly on the embedded hardware. The inference time, denoted as DT in the UART output, represents the number of milliseconds required for the neural network to process one image on the STM32.

8. Experimental Results

The Python-side experiments showed that all four networks were capable of achieving high accuracy on MNIST, as confirmed by their confusion matrices. However, when deployed on the STM32, only MobileNetV2 and ResNet50 could be evaluated. MobileNetV2 provided a good balance between speed and accuracy, while ResNet50 achieved very high accuracy but required significantly more computation time. In the final hardware-in-the-loop experiments, ResNet50 reached an accuracy of approximately 98% over 100 randomly selected test samples, with an average inference time of around 120 ms per image. These results demonstrate that even

relatively large deep networks can be executed on microcontrollers when proper quantization and optimization techniques are applied.

STM32 real-time inference output for the deployed INT8 model over a serial (COM) interface. Each line shows the true MNIST label sent from the host PC, the predicted class returned by the STM32 board, and whether the prediction was correct. This output confirms correct end-to-end communication and functional on-device inference.

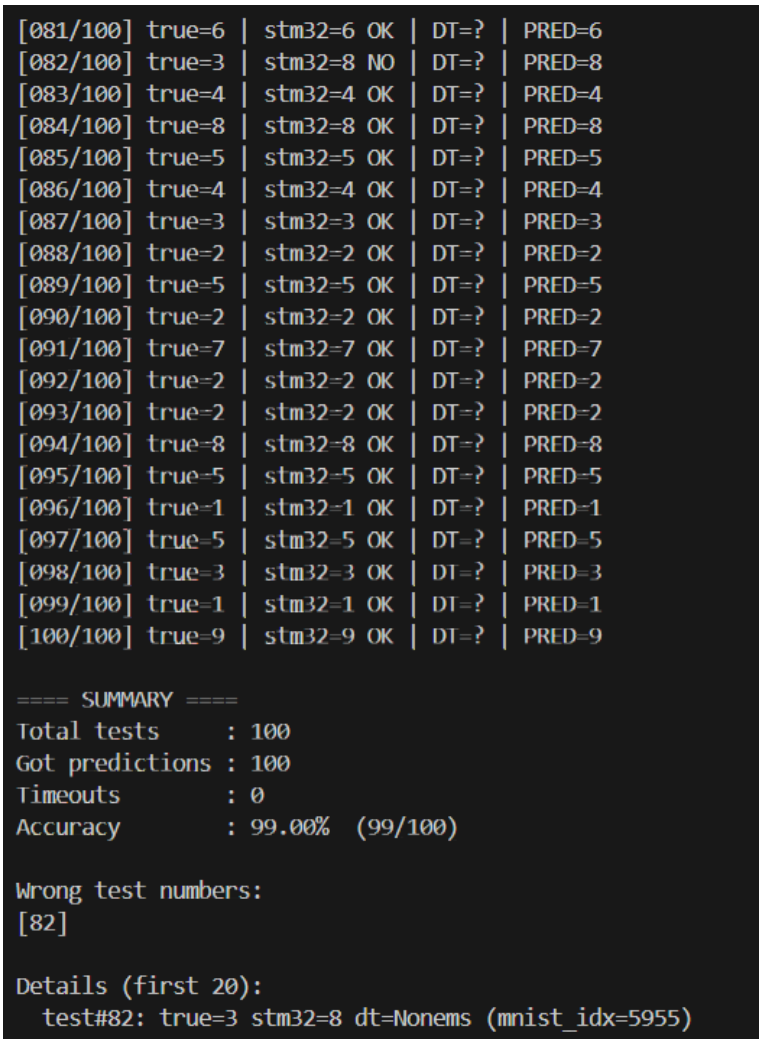

Figure 5 - STM32 Real-Time Inference Log (No Timing) for MobileNet_v2 Model

STM32 real-time inference results including per-image inference latency (DT). The measured DT values, approximately 119–120 ms per image, represent the end-to-end processing time for image reception, preprocessing, neural network inference, and result transmission. This demonstrates the real-time capability of the deployed quantized neural network on the STM32 microcontroller.

```
[081/100] true=6 | stm32=6 OK | DT=119ms | PRED=6 DT=119
[082/100] true=3 | stm32=8 NO | DT=120ms | PRED=8 DT=120
[083/100] true=4 | stm32=4 OK | DT=120ms | PRED=4 DT=120
[084/100] true=8 | stm32=8 OK | DT=120ms | PRED=8 DT=120
[085/100] true=5 | stm32=5 OK | DT=119ms | PRED=5 DT=119
[086/100] true=4 | stm32=4 OK | DT=120ms | PRED=4 DT=120
[087/100] true=3 | stm32=3 OK | DT=120ms | PRED=3 DT=120
[088/100] true=2 | stm32=2 OK | DT=119ms | PRED=2 DT=119
[089/100] true=5 | stm32=5 OK | DT=120ms | PRED=5 DT=120
[090/100] true=2 | stm32=2 OK | DT=120ms | PRED=2 DT=120
[091/100] true=7 | stm32=7 OK | DT=119ms | PRED=7 DT=119
[092/100] true=2 | stm32=2 OK | DT=120ms | PRED=2 DT=120
[093/100] true=2 | stm32=2 OK | DT=119ms | PRED=2 DT=119
[094/100] true=8 | stm32=8 OK | DT=120ms | PRED=8 DT=120
[095/100] true=5 | stm32=5 OK | DT=120ms | PRED=5 DT=120
[096/100] true=1 | stm32=1 OK | DT=120ms | PRED=1 DT=120
[097/100] true=5 | stm32=5 OK | DT=119ms | PRED=5 DT=119
[098/100] true=3 | stm32=3 OK | DT=120ms | PRED=3 DT=120
[099/100] true=1 | stm32=1 OK | DT=120ms | PRED=1 DT=120
[100/100] true=9 | stm32=9 OK | DT=120ms | PRED=9 DT=120

==== SUMMARY ====
Total tests     : 100
Got predictions : 100
Timeouts        : 0
Accuracy        : 98.00%  (98/100)

Wrong test numbers:
[7, 82]

Details (first 20):
  test#7: true=6 stm32=4 dt=120ms (mnist_idx=625)
  test#82: true=3 stm32=8 dt=120ms (mnist_idx=5955)
```

Figure 6 - STM32 Inference with Latency Measurement for ResNet50 Model

9. Neural Network Model and Training

The experiments highlight the fundamental trade-off between model complexity, memory usage and inference speed in embedded artificial intelligence. While SqueezeNet and EfficientNet-B0 are designed to be lightweight, their memory footprint after quantization was still too large for the STM32 target. MobileNetV2 was specifically designed for embedded deployment and therefore showed robust performance within the hardware limits. ResNet50, despite being significantly deeper, was still able to run after INT8 quantization, demonstrating the effectiveness of modern model compression techniques. The small accuracy loss introduced by quantization was negligible compared to the large gains in memory and computational efficiency.

10.Conclusion

This project successfully demonstrated the complete pipeline of training, quantizing, deploying and evaluating deep neural networks on an STM32 microcontroller. By combining modern CNN architectures with INT8 quantization and the X-CUBE-AI framework, it was possible to run real-time handwritten digit recognition on a highly constrained embedded system. The hardware-in-the-loop testing framework provided a realistic measure of performance and validated that high-accuracy deep learning models can be deployed on microcontrollers when properly optimized. This work illustrates the practical feasibility of embedded artificial intelligence for real-world applications such as vision, sensing and edge computing.